

Transferencia -> Transacción atómica

- Tiene \$ Cuenta A?
- Resta cuenta A
- \\
- Suma cuenta B

Tres pasos: Existencia, crédito, Débito

Commit/Rollback: Modificaciones en la BD

Propiedades **ACID** (no necesariamente resuelve deadlocks o errores)

- Atomicidad (o se hace o no)
- Consistencia
- Aislamiento
- Durabilidad

Mirada optimista/pesimista

- Entrelazados (Errores) (lectura sucia, overwriting)

Anomalia

**Schedules** (planes de ejecución)  
(Shared-Lock Architecture)

o

Exclusive-Lock

**Regla Estricta** (lock retenido hasta transacción)

DEADLOCKS (bloqueo doble/mutuo)

**Prevención:**

- S2PL Conservador
- Ordenamiento Total de Recursos
- Basados en timestamps (Wait-Die o Wound-Wait)

Detección: Grafo de espera detecta ciclo (Transacción víctima: por Inanición o por Reinicio Cíclico)

Concurrencia (se usan logs para control)

**Modelo X/Open XA**

- Application Program (AP)
- Transaction Manager (TM)
- Resource Manager (RM)

- Communications Resource Manager (CRM)

### **TWO PHASE COMMIT 2PC** (con coordinador)

- Fase 1: Petición (Prepare)
  - Voto positivo/negativo
- Fase 2: Decisión (Commit/Abort)

PROBLEMA: Transacciones en duda

SOLUCIONES:

Commit en Dos Fases en Árbol (si uno aborta, se aborta todo)

Commit en Dos Fases Dinámico (sin coordinador fijo)

### **COMMIT EN 3 FASES (3PC)**

Previene bloqueos por caída de coordinador

+ Máquina de estados + fase intermedia

Problema: Partición de Red (si se corta la red, cae todo). Más complejo, pide más disponibilidad de recursos

Ricardo:

**Transacción distribuida:** Gestor de colas transaccional (para “rollback” de un envío de correo)